Java Algorithms

Built-in Intelligence for Data Structures

CODEHIVE

1. Introduction to Collections Algorithms

Algorithms are procedural blueprints used to solve computational problems. In Java, you don't always have to write these from scratch. The `Collections` utility class (part of `java.util`) provides a robust suite of static methods designed to sort, search, and shuffle your data.

> **Key Rule:** Most algorithms in the Collections class work specifically on `List` types (like ArrayList or LinkedList) because they rely on positional indexing.

2. Strategic Sorting

Sorting is the foundation of data organization. Java uses a variation of TimSort, which is incredibly efficient for real-world data.

```
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(5);
numbers.add(1);
numbers.add(9);

// Natural Ascending Order [1, 5, 9]
Collections.sort(numbers);

// Descending Order [9, 5, 1]
Collections.sort(numbers, Collections.reverseOrder());
```

3. High-Speed Searching

While you can find an item by looping (Linear Search), Java provides the `binarySearch()` method for massive performance gains on large datasets.

Crucial Requirement: The list MUST be sorted before you call `binarySearch()`. If the list is unsorted, the result is undefined.

```
Collections.sort(names);
int index = Collections.binarySearch(names, "Angie");

// O(log n) efficiency: Fast even for millions of rows.
```

4. Shuffling & Randomization

Whether you are building a card game or randomizing a quiz, the `shuffle()` algorithm is the industry standard for reordering elements randomly.

```
ArrayList<String> deck = new ArrayList<>();
deck.add("Ace");
deck.add("King");
deck.add("Queen");

// Randomizes the order in-place
Collections.shuffle(deck);
```

5. Extremes: Min and Max

Finding the largest or smallest element in a collection doesn't require a loop. Java can scan the collection and return the extreme values based on their natural order.

```
ArrayList<Integer> scores = new ArrayList<>();
scores.add(450);
scores.add(120);
scores.add(890);

int top = Collections.max(scores); // 890
int low = Collections.min(scores); // 120
```

6. Frequency & Occurrences

Need to know how many times a specific object appears? The `frequency()` method handles this and works on ALL collection types (not just Lists).

```
ArrayList<String> votes = new ArrayList<>();
votes.add("Yes");
votes.add("No");
votes.add("Yes");

int result = Collections.frequency(votes, "Yes"); // 2
```

7. Swapping Elements

Manually swapping two elements in a list requires a temporary variable and three lines of code. The `swap()` algorithm reduces this to one readable line.

```
// Swapping index 0 with index 2
Collections.swap(myList, 0, 2);
```

This is extremely useful when implementing custom sorting logic or reordering UI elements.

8. The Iterator Pattern

Algorithms rely on "Iteration." You can choose between the clean **For-Each** style or the explicit **Iterator** style.

For-Each:
`for(String s : list)`
Best for: Read-only access and clean syntax.

Iterator:
`while(it.hasNext())`
Best for: Modifying or removing items safely.

9. Summary & Final Verdict

- Algorithms are procedures to solve problems efficiently.
- Java provides built-in tools in the `Collections` class.
- **Binary Search** requires prior **Sorting**.
- Utility methods like `shuffle`, `frequency`, and `swap` prevent "reinventing the wheel."